

list 정리

이중 연결 리스트 기반 컨테이너로 중간 삽입/삭제에 강합니다.

사용처 (Where)

- 중간 위치 삽입/삭제가 자주 일어날 때
- iterator를 유지하며 원소를 옮길 때
- 연결 리스트 개념을 STL로 다룰 때

방법 (How to Use)

- push_back/front로 추가
- iterator로 위치 이동
- insert/erase로 중간 처리
- sort는 li.sort() 사용

기본 정보

- 필요 헤더: #include <list>
- 기본 선언: list<int> li;

왜 써야 할까? (Why)

- 직접 포인터로 연결 리스트를 구현하지 않아도 됨
- 삭제/삽입 과정에서 포인터 실수를 줄임
- 연결 리스트 원리를 실습하기 좋음

주요 함수

함수/문법	의미
push_back/front	양끝 추가
insert(it,x)	it 위치 앞에 삽입
erase(it)	it 위치 삭제
sort()	리스트 정렬
remove(x)	값이 x인 원소 제거

기본 코드 예시

```
#include <iostream>
#include <list>
using namespace std;

int main() {
    list<int> li = {1, 3, 4};
    auto it = li.begin();
    ++it;
    li.insert(it, 2);

    for (int x : li) cout << x << ' ';
}
```

list 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
양끝 삽입/삭제	$O(1)$
중간 삽입/삭제	위치 알면 $O(1)$
검색	$O(n)$
인덱스 접근	지원 안 함

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	이미 구현된 기능 사용	자료구조 설계와 예외 처리 필요	배열/포인터 직접 관리
코드 길이	짧고 읽기 쉬움	길고 버그 가능성 증가	간단하지만 확장성이 낮음
성능	일반 상황에서 충분히 최적화	구현 실력에 따라 달라짐	고정 크기에서는 단순함
안정성	범위/메모리 관리 부담 감소	메모리/인덱스 실수 가능	범위 초과 위험 큼
추천 상황	대부분의 C++ 코드	자료구조 원리 학습	아주 단순한 고정 크기 데이터

응용: 조건에 맞는 값 삭제

iterator 기반으로 순회하면서 특정 조건의 원소를 안전하게 삭제합니다.

```
#include <iostream>
#include <list>
using namespace std;

int main() {
    list<int> li = {1, 2, 3, 4, 5};

    for (auto it = li.begin(); it != li.end(); ) {
        if (*it % 2 == 0) it = li.erase(it);
        else ++it;
    }

    for (int x : li) cout << x << ' ';
}
```

처음 배울 때 자주 하는 실수

- `li[i]`처럼 인덱스 접근 불가
- 검색은 `find`
- 일반적인 저장/정렬은 `vector`가 더 자주 쓰임

비슷한 항목과 차이

- vector는 인덱스 접근과 정렬에 강함
- deque는 양끝 삽입/삭제에 강함
- forward_list는 단일 연결 리스트

한 줄 정리: list은(는) '중간 위치 삽입/삭제가 자주 일어날 때' 상황에서 먼저 떠올리면 좋은 STL입니다.